



NAME: A Roja

REG NO: 192424226

COURSE CODE: DSA0305 Natural Language Processing

- 1 A multinational recruitment company receives thousands of resumes every day in text format. The HR department requires an automated system to extract candidate information for shortlisting.**

Design and implement a Python-based resume information extraction system using Regular Expressions that performs the following tasks:

Extract the candidate's name.

Identify email addresses.

Extract mobile numbers.

Detect technical skills (Python, Java, SQL, Machine Learning, NLP).

Extract years of experience.

Generate a structured summary of the candidate's profile.

Display candidates who satisfy the minimum eligibility criteria (minimum 2 years of experience and Python skill).

Aim

To develop a Python program using Regular Expressions to automatically extract candidate information from resumes and identify eligible candidates.

Algorithm

Import the re module.

Read the resume text.

Extract the candidate name.

Extract email address.

Extract mobile number.

Search for technical skills.

Extract years of experience.

Display candidate profile.

Check eligibility:

Experience $\geq$ 2 years

Python skill available

Display eligibility result.

CODE:

```
import re
resume = """
Name: Rahul Sharma
Email: rahul@gmail.com
Mobile: 9876543210
Skills: Python, Java, SQL, Machine Learning
Experience: 3 years
"""

# Name
name = re.search(r"Name:\s*(.*)", resume)
name = name.group(1)

# Email
email = re.search(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", resume)
email = email.group()

# Mobile
mobile = re.search(r"\b\d{10}\b", resume)
mobile = mobile.group()

# Skills
skills = ["Python", "Java", "SQL", "Machine Learning", "NLP"]

found_skills = []

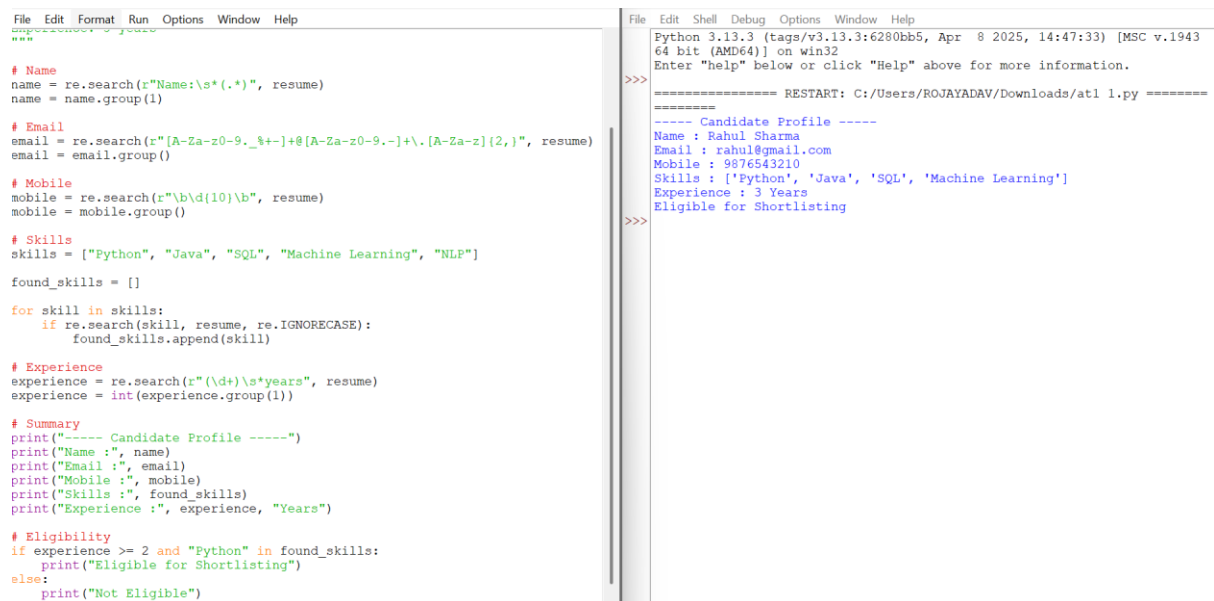
for skill in skills:
    if re.search(skill, resume, re.IGNORECASE):
        found_skills.append(skill)
```

```
# Experience
experience = re.search(r"(\d+)\s*years", resume)
experience = int(experience.group(1))
```

```
# Summary
print("----- Candidate Profile -----")
print("Name :", name)
print("Email :", email)
print("Mobile :", mobile)
print("Skills :", found_skills)
print("Experience :", experience, "Years")
```

```
# Eligibility
if experience >= 2 and "Python" in found_skills:
    print("Eligible for Shortlisting")
else:
    print("Not Eligible")
```

OUTPUT:



```
File Edit Format Run Options Window Help
#####
# Name
name = re.search(r"Name:\s*(.*)", resume)
name = name.group(1)

# Email
email = re.search(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", resume)
email = email.group()

# Mobile
mobile = re.search(r"\b\d{10}\b", resume)
mobile = mobile.group()

# Skills
skills = ["Python", "Java", "SQL", "Machine Learning", "NLP"]
found_skills = []
for skill in skills:
    if re.search(skill, resume, re.IGNORECASE):
        found_skills.append(skill)

# Experience
experience = re.search(r"(\d+)\s*years", resume)
experience = int(experience.group(1))

# Summary
print("----- Candidate Profile -----")
print("Name :", name)
print("Email :", email)
print("Mobile :", mobile)
print("Skills :", found_skills)
print("Experience :", experience, "Years")

# Eligibility
if experience >= 2 and "Python" in found_skills:
    print("Eligible for Shortlisting")
else:
    print("Not Eligible")

File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943
64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ROJAYADAV/Downloads/at1 1.py =====
----- Candidate Profile -----
Name : Rahul Sharma
Email : rahul@gmail.com
Mobile : 9876543210
Skills : ['Python', 'Java', 'SQL', 'Machine Learning']
Experience : 3 Years
Eligible for Shortlisting
>>>
```

2 An e-commerce company plans to enhance its product search engine to improve customer experience through flexible keyword matching.

Design and implement a Python-based product search system using Regular

Expressions that performs the following tasks:

Search products using an exact keyword.

Perform prefix-based searches.

Perform suffix-based searches.

Search using partial keywords.

Perform case-insensitive searches.

Display all matching product names.

Generate a report showing the total number of matching products for each search.

Aim

To develop a product search system using Regular Expressions for exact, prefix, suffix, partial, and case-insensitive searches.

Algorithm

Import re.

Create a product list.

Accept search keyword.

Perform:

Exact search

Prefix search

Suffix search

Partial search

Case-insensitive search

Display matching products.

Display total matches.

CODE:

```
import re

products = [
    "Apple Laptop",
    "Apple Mobile",
    "Samsung Mobile",
    "Dell Laptop",
    "HP Laptop",
    "Boat Earphones",
    "Sony Headphones",
    "LG Television"
]
keyword = input("Enter keyword: ")

# Exact Search
exact = [p for p in products if re.fullmatch(keyword, p)]

# Prefix Search
```

```

prefix = [p for p in products if re.match(keyword, p)]

# Suffix Search
suffix = [p for p in products if re.search(keyword + r"$", p)]

# Partial Search
partial = [p for p in products if re.search(keyword, p)]

# Case-insensitive Search
ignore = [p for p in products if re.search(keyword, p, re.IGNORECASE)]

print("\nExact Search:", exact)
print("Prefix Search:", prefix)
print("Suffix Search:", suffix)
print("Partial Search:", partial)
print("Case Insensitive:", ignore)

print("\nSearch Report")
print("Exact Matches :", len(exact))
print("Prefix Matches :", len(prefix))
print("Suffix Matches :", len(suffix))
print("Partial Matches :", len(partial))
print("Case Insensitive Matches :", len(ignore))

```

OUTPUT:

```

File Edit Format Run Options Window Help
products = [
    "Apple Laptop",
    "Apple Mobile",
    "Samsung Mobile",
    "Dell Laptop",
    "HP Laptop",
    "Boat Earphones",
    "Sony Headphones",
    "LG Television"
]
keyword = input("Enter keyword: ")

# Exact Search
exact = [p for p in products if re.fullmatch(keyword, p)]

# Prefix Search
prefix = [p for p in products if re.match(keyword, p)]

# Suffix Search
suffix = [p for p in products if re.search(keyword + r"$", p)]

# Partial Search
partial = [p for p in products if re.search(keyword, p)]

# Case-insensitive Search
ignore = [p for p in products if re.search(keyword, p, re.IGNORECASE)]

print("\nExact Search:", exact)
print("Prefix Search:", prefix)
print("Suffix Search:", suffix)
print("Partial Search:", partial)
print("Case Insensitive:", ignore)

print("\nSearch Report")
print("Exact Matches :", len(exact))
print("Prefix Matches :", len(prefix))
print("Suffix Matches :", len(suffix))

```

```

Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943
64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ROJAYADAV/Downloads/at1 l.py =====
Enter keyword: laptop

Exact Search: []
Prefix Search: []
Suffix Search: []
Partial Search: []
Case Insensitive: ['Apple Laptop', 'Dell Laptop', 'HP Laptop']

Search Report
Exact Matches : 0
Prefix Matches : 0
Suffix Matches : 0
Partial Matches : 0
Case Insensitive Matches : 3
>>>

```

3 A university is developing an automated online registration portal to verify student information before course enrollment.

Design and implement a Python-based validation system using Regular Expressions that performs the following tasks:

Validate the student register number.

Validate the institutional email address.

Validate the course code.

Validate the semester information.

Validate the student's mobile number.

Display appropriate validation messages for each field.

Generate a final registration status report indicating whether the registration is successful.

Aim

To validate student registration details using Regular Expressions.

Algorithm

Import re.

Read:

Register Number

Email

Course Code

Semester

Mobile Number

Validate each field.

Display validation result.

If all validations pass, display Registration Successful.

CODE:

```
import re
```

```
reg = input("Register Number: ")
email = input("Institution Email: ")
course = input("Course Code: ")
semester = input("Semester: ")
mobile = input("Mobile Number: ")
```

```
valid = True
```

```
# Register Number
if re.fullmatch(r"\d{12}", reg):
    print("Register Number Valid")
else:
    print("Invalid Register Number")
    valid = False
```

```

# Email
if re.fullmatch(r"[A-Za-z0-9._%+-]+@saveetha\.com", email):
    print("Email Valid")
else:
    print("Invalid Email")
    valid = False

# Course Code
if re.fullmatch(r"[A-Z]{3}\d{2}", course):
    print("Course Code Valid")
else:
    print("Invalid Course Code")
    valid = False

# Semester
if re.fullmatch(r"[1-8]", semester):
    print("Semester Valid")
else:
    print("Invalid Semester")
    valid = False

# Mobile
if re.fullmatch(r"[6-9]\d{9}", mobile):
    print("Mobile Number Valid")
else:
    print("Invalid Mobile Number")
    valid = False

if valid:
    print("\nRegistration Successful")
else:
    print("\nRegistration Failed")

```

OUTPUT:

```

File Edit Format Run Options Window Help
valid = True

# Register Number
if re.fullmatch(r"\d{12}", reg):
    print("Register Number Valid")
else:
    print("Invalid Register Number")
    valid = False

# Email
if re.fullmatch(r"[A-Za-z0-9._%+-]+@saveetha\.com", email):
    print("Email Valid")
else:
    print("Invalid Email")
    valid = False

# Course Code
if re.fullmatch(r"[A-Z]{3}\d{2}", course):
    print("Course Code Valid")
else:
    print("Invalid Course Code")
    valid = False

# Semester
if re.fullmatch(r"[1-8]", semester):
    print("Semester Valid")
else:
    print("Invalid Semester")
    valid = False

# Mobile
if re.fullmatch(r"[6-9]\d{9}", mobile):
    print("Mobile Number Valid")
else:
    print("Invalid Mobile Number")
    valid = False

if valid:
    print("\nRegistration Successful")

```

```

File Edit Shell Debug Options Window Help
Python 3.13.3 (tags/v3.13.3:6280bb5, Apr 8 2025, 14:47:33) [MSC v.1943
64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
===== RESTART: C:/Users/ROJAYADAV/Downloads/at1 l.py =====
Register Number: 192424226
Institution Email: roja226@gmail.com
Course Code: DSA0305
Semester: 3
Mobile Number: 9876543210
Invalid Register Number
Invalid Email
Invalid Course Code
Semester Valid
Mobile Number Valid
Registration Failed
>>>
===== RESTART: C:/Users/ROJAYADAV/Downloads/at1 l.py =====
Register Number: 1923456789012
Institution Email: roja@saveetha.com
Course Code: DSA03
Semester: 3
Mobile Number: 9876543210
Invalid Register Number
Email Valid
Course Code Valid
Semester Valid
Mobile Number Valid
Registration Failed
>>>

```

CONCLUSION:

The three Python programs successfully demonstrate the application of Regular Expressions (Regex) in solving real-world Natural Language Processing tasks. The Resume Information Extraction System efficiently extracts candidate details and identifies eligible applicants, the Product Search System provides flexible and accurate product searching using different regex patterns, and the University Registration System validates student information before registration. These programs improve automation, reduce manual effort, ensure data accuracy, and showcase the practical use of Regular Expressions in information extraction, pattern matching, and input validation.